

ProFi: Scalable and Efficient Website Fingerprinting

Patrick Krämer¹, Member, IEEE, Benedikt Baier², Niklas Landerer,

Philip Diederich³, Graduate Student Member, IEEE, Alexander Griessel¹, Graduate Student Member, IEEE,

Oliver Hohlfeld, Andreas Blenk¹, Martin Mieth, and Wolfgang Kellerer¹, Senior Member, IEEE

Abstract—Website Fingerprinting (WFP) attacks infer the websites or webpages a user is visiting from encrypted traffic. To date, it remains uncertain if WFP can attack many users from a central location in an online scenario. We close this gap with PROFI, a WFP attack that detects websites based on the initial TLS connection from the client to the server using at most the connection's first 30 packets. PROFI achieves a precision and recall of 86.51 % and 85.35 % in a closed-world, and 68.90 % and 78.71 % in an open-world scenario, which is competitive to state-of-the-art (SoA) WFP attacks, while taking a fraction of the time of SoA attacks to classify a webpage. Further, we implement PROFI as a micro service-based prototype and evaluate the attack in an online scenario with real traffic traces. We show that PROFI can monitor up to 100 websites at 10G, corresponding to up to 424 webpages per second. We also show that PROFI has the potential to interfere with a victim's webpage access.

Index Terms—Website fingerprinting, machine learning, privacy, time series modeling, probabilistic graphical models.

I. INTRODUCTION

ENCRYPT every transfer [1] is a major step towards securing the privacy of Internet users: DNS over TLS¹ (DoT) [2], and DNS² over HTTPS³ (DoH) [3] secure the user's DNS queries; HTTPS with the Encrypted Server Name Indication (ESNI) extension [4], and its successor, the Encrypted Client Hello (ECH) extension [5], secure the communication between clients and remote servers. Hence, adversaries, legitimate or not, can no longer analyze DNS traffic, packet payload, the Server Name Indication (SNI), or the information in the TLS Application-Layer Protocol

Negotiation (ALPN) extension [6]. This poses a severe problem to censors since encrypting every transfer forces them to choose between censoring all traffic or none [7]. Censoring all traffic, as done before [8], can be expensive and severe ties to large portions of the Internet. In 2019, at least 10 % of the Alexa top 1 Million websites supported ESNI and would have been unavailable [7]. Thus, adversaries must rely on other methods to analyze users' communication. We thus believe that Website Fingerprinting (WFP) [9] will become relevant beyond anonymity networks such as the The Onion Router (Tor) network. WFP allows adversaries to infer visited websites from encrypted traffic using patterns in the data exchange between client and server.

However, launching WFP attacks at scale in the network is challenging. The attack is likely to occur at a location that allows the monitoring of multiple users, i.e., the attack must operate live in the network at higher link rates (say 10Gbit/s) on a traffic aggregate. Previous attacks [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35] are not applicable in this scenario. Previous attacks are evaluated offline, target individual users, and use Machine Learning (ML) models that take milliseconds to evaluate and thus do not scale to high-volume links. Further, most previous attacks use features calculated from entire page loads, i.e., require extracting individual page loads from the traffic aggregate. Extracting page loads is difficult considering that simply detecting the onset of one concurrent page load in a single user scenario is already challenging [19], [22], [29], [36]. Thus, it remains unclear if WFP poses a threat beyond the surveillance of individuals. Consequently, the goal of this work is to design, implement and evaluate a WFP attack that can operate live in the network to establish the potential threat WFP poses to general Internet users.

To show that WFP is a threat in a general setting, we design, implement, realize, and evaluate PROFI (PRObabilistic FInderprinting). In contrast to previous work, PROFI's design includes three aspects beyond the mere detection rate: Data availability during operation, inference speed, and continuous operation. PROFI uses Probabilistic Graphical Models (PGMs) [37] to model the initial TLS connection established during webpage retrieval based on the direction, size, and TLS records among the connection's first 30 packets, which are, as we show in a prototypical implementation, readily obtainable in practice. Contrary to previous attacks, PROFI handles traffic aggregates without further processing because of the flow-based classification. PROFI shows competitive performance

Manuscript received 1 September 2022; revised 6 February 2023 and 23 May 2023; accepted 13 September 2023. Date of publication 25 September 2023; date of current version 7 February 2024. This work is partially funded by the German Research Foundation (DFG) as part of the ADVISE project (grant ID - 438892507) and the SDN APP AWARE project (grant ID - 316878574), and by the Germany Federal Ministry of Education and Research (BMBF) as a part of the SoftwareCampus initiative (grant ID 01IS17049) and the ALNET PROTECT project (grant ID 16KIS1318). The associate editor coordinating the review of this article and approving it for publication was L. Y. Chen. (Corresponding author: Patrick Krämer.)

Patrick Krämer, Benedikt Baier, Niklas Landerer, Philip Diederich, Alexander Griessel, and Wolfgang Kellerer are with the Chair of Communication Networks, Technical University of Munich, 80333 Munich, Germany (e-mail: patrick.kraemer@tum.de).

Oliver Hohlfeld is with the Distributed Systems Group, University of Kassel, 34127 Kassel, Germany.

Andreas Blenk is with the T CED INW-DE, Siemens AG, 81739 Munich, Germany.

Martin Mieth is with the Deep Packet Inspection Development, ipoque GmbH - A Rohde & Schwarz Company, 04109 Leipzig, Germany.

Digital Object Identifier 10.1109/TNSM.2023.3318508

¹Transport Layer Security (TLS).

²Domain Name Service (DNS).

³Hyper Text Transfer Protocol Secure (HTTPS).

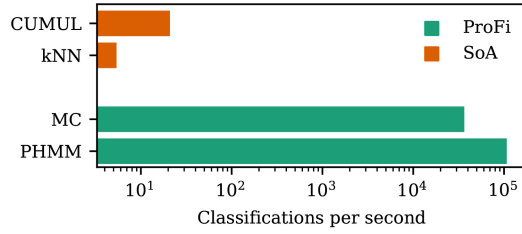


Fig. 1. Classifications per second that PROFi and state-of-the-art (SoA) approaches achieve in an offline setting on one Central Processing Unit (CPU) core. Values are obtained for models from Section IV-D on our data described in Section V-A.

to three state-of-the-art WFP attacks, achieving precision and recall of 86.51 % and 85.35 % in a closed-world, and 68.90 % and 78.71 % in an open-world scenario. Further, the PGMs have few parameters, enabling inference within microseconds. In contrast to previous attacks, PROFi's ML models allow PROFi to classify thousands of webpages per second, as Fig. 1 shows. The PGMs further simplify the continuous operation of the attack since the PGMs readily indicate data drift through their log-likelihood. Other ML models require more involved monitoring, e.g., by observing and evaluating samples during production [38]. Further, the PGMs enable a modular microservice-based implementation amenable to deployment on top of, e.g., Multi-Access Edge Computing (MEC) [39] infrastructure. We evaluate the performance of the resulting system in testbed measurements. We show that PROFi's microservice-based implementation can handle more than 400 webpage accesses per second on a 10 G link at full rate. Overall, our contributions are as follows:

- Collection of an extensive dataset of traffic from 4 800 webpages of 96 websites collected over 70 days.
- Implementation of a TLS dissector for feature extraction.
- Design, implementation, and evaluation of PROFi, a PGM-based WFP attack that is designed for the operation live in the network.
- A microservice-based testbed implementation and evaluation of PROFi.
- We show that PROFi can handle 100s of website accesses per second. This is the first implementation and evaluation of a WFP attack's potential to operate live in the network and has relevance beyond WFP, i.e., extends to ML-based traffic classification in general.
- The design of a readily-implementable defense against PROFi reducing precision and recall to less than 10 % and 20 % while causing a bandwidth overhead of 150 %.

The remainder is organized as follows. Section II introduces the attack scenario, Section III introduces background information and related work. Section IV presents the design of the PROFi attack and the defense method. Section V introduces the dataset and analyzes the training process. Section VI evaluates the classification performance of the PROFi attack and compares PROFi to three baselines. Section VII presents the microservice-based implementation of PROFi, and evaluates its throughput, scalability, and labeling speed. Section VIII discusses ethical considerations, and Section IX concludes this paper.

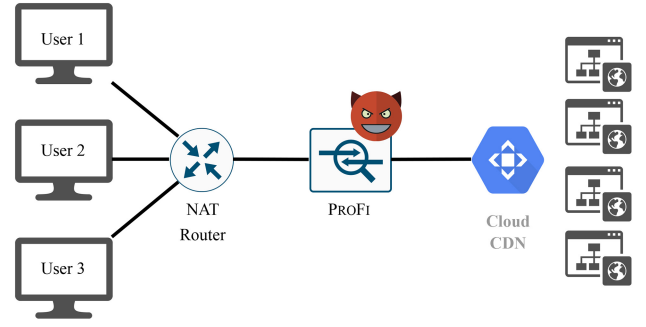


Fig. 2. PROFi attack scenario.

II. ATTACK SCENARIO

Fig. 2 illustrates the attack scenario of PROFi: Multiple users share a common access point to the Internet—a typical Internet access scenario (e.g., home network or public WiFi).

The user view: The users are situated behind a Network Address Translation (NAT)-capable router, as in a typical home or campus network, which renders a realistic attack scenario. The users share the same public Internet Protocol (IP) address; hence, they cannot be differentiated by their IP addresses. Since most of today's Web traffic is carried by large Content Delivery Networks (CDNs) [40], [41], [42], we focus on a scenario where users retrieve webpages from CDNs. We focus on CDNs since CDNs serve many popular websites [30], and websites can use CDNs to hide from censors [43]. Since a CDN server can deliver any website hosted by the CDN, this makes the attack more challenging since the server's IP address does not identify one website. We do not investigate the effect of Virtual Private Networks (VPNs) on the attack. More than two third of Internet users do not use a VPN [44].

The attacker view: The goal of the attacker (e.g., a censor) is to discern the website a user is accessing, where a website consists of multiple webpages. The attacker is situated on the path between the NAT router and the CDN. Contrary to previous WFP attacks, the attacker has no access to the private network of the users. However, the attacker still has access to traffic in both directions. The traffic is encrypted with TLS, and the SNI and ALPN extensions in the TLS handshake are also encrypted. This prevents usage of either feature for fingerprinting [8]. Further, the attacker knows only the public IP of the NAT router for every user. Furthermore, as websites are hosted on CDNs, attackers cannot fingerprint websites based on the IP addresses of destination (cache or edge) servers. Thus, we investigate a scenario where the attacker is left with statistical information about the frequency, volume, and direction of data transfers within a flow.

Per-Flow Classification: Our attack classifies based on the first TLS connection the user's client establishes to retrieve a webpage. Our attack uses observable features of the traffic, such as the frame size, TLS record size, and the direction of packets. The underlying assumption that we confirm in our evaluation is that the server and client exchange unique information in the initial flow, which reflects in a specific pattern of exchanged packets in the network. Thus, the attack can classify each flow individually, sidestepping the extraction

of individual webpage calls from the traffic aggregates, as previous attacks require, which is very challenging to perform in practice. Previous work shows that even solely identifying the *start* of overlap for two overlapping webpage calls of a single user is difficult [19], [22], [29], [36].

Practical feasibility: Since we want to investigate whether WFP is a threat to the general public, the attack must be able to process large traffic volumes, requiring fast and efficient WFP mechanisms. Further, the attack must be deployable and continuously operational, i.e., we consider not only training and evaluation but also the full life cycle of an ML application [38]. Therefore, we decided against more complex models like Random Forests, Support Vector Machines (SVMs), and Deep Neural Networks (DNNs).⁴ Specifically, there is a trend of using DNNs for WFP [23], [24], [27], [29], [34]. However, these DNNs are large, with many layers and parameters. For example, Var-CNN [27] uses two Convolutional Neural Networks with 18 layers and an input layer with 5 000 neurons each. Despite their size, simpler ML models can outperform DNNs on traffic classification tasks [45]. Further, executing DNNs requires specialized hardware and a specifically designed system architecture to operate online in the network [46]. Further, DNNs are susceptible to shortcut learning, and results are difficult to verify [47]. Thus, we focus on simple ML models that run on commodity hardware and are easy to interpret. This makes the development and deployment of WFP attack faster and the attack more severe, but also helps in understanding WFP attacks to mitigate them.

III. BACKGROUND AND RELATED WORK

WFP infers the websites a user is accessing from passive network traffic observations [12].

Website vs. webpage: Attacks can infer *websites* and *webpages* [13]. A *webpage* is identified by a specific Unified Resource Locator (URL). A *website* consists of multiple webpages below a Second Level Domain (SLD). For example, www.nih.gov/grants-funding and www.nih.gov/research-training are two webpages from one website. Related work often performs WFP on the level of webpages, e.g., the landing page—here, the attacker has to learn a model that recognizes only this single page. This task is easier than learning a model for arbitrary webpages of one website. A website’s webpages show different content, resulting in different packet sequences. Thus, website fingerprinting has to cope with a larger variance in the data compared to webpage fingerprinting [13], making the data more difficult to model [48], [49]. PROFi performs the more challenging website fingerprinting attack, i.e., PROFi extracts patterns that generalize from examples of webpages of websites to the traffic of unseen webpages of that site.

Open vs. closed world: Two evaluation scenarios exist for WFP attacks: the closed-world and the open-world scenario [12]. In the closed-world scenario, the attacker knows

the websites a victim visits. That is, the attacker can train the attack model for every visited website—no other websites are visited. This is the easiest variant of the attack. In the more realistic open-world scenario, the attacker does not know which websites the victim access. That is, the victim also visits websites that were not included in the attacker’s training data. Generally, the open-world scenario is considered the more challenging scenario. We evaluate both scenarios.

WFP attacks: Many recent attacks perform *webpage* fingerprinting, e.g., try to detect the landing pages of websites [9], [11], [12], [15], [16], [18], [19], [21], [22], [23], [24], [25], [27], [28], [29], [30], [31], [32], [33], [34], [36], [50], [51], [52]. Fewer works perform the more challenging *website* fingerprinting [13], [20], [26], [35]. Except for [18], [19], [22], [26], [35], [36], previous work makes the single-page-load assumption [19], i.e., users load only one webpage and there is no background traffic. All previous work evaluates their attacks under a single-user assumption, i.e., only one active user or process generates traffic. Closest to our approach is the work of Shen et al. [28], Zhuo et al. [26], Hoang et al. [30], and Trevisan et al. [35]. Shen et al. classify normal TLS traffic from one website and use only traffic from TLS sessions with the SNI of the top-level domain, the first 100 packets of a webpage load and the kNN classifier. Zhuo et al. use a Profile Hidden Markov Model (PHMM) and consider subsequent webpage accesses of a single user. Hoang et al. build fingerprints from the IP addresses of webpage calls and is thus computationally efficient. Trevisan et al. [35] use per-flow features and a Random Forest classifier to label flows with domain names. In contrast, PROFi uses a novel WFP classification method based on PGMs and anomaly detection that makes PROFi easy to train and execute. The main contribution of this work is the evaluation of the attack in a testbed, i.e., the investigation if and at what scale WFP can be executed online in the network and interfere with user traffic. PROFi is different from previous WFP attacks on the Tor network in the following ways. PROFi makes predictions for flows and does not need access to all packets of each webpage call. Further, PROFi does not assume the monitoring of an individual user.

WFP defenses: A number of mechanisms to mitigate WFP attacks exists [14], [17], [31], [53], [54], [55], [56], [57], [58], [59], [60], [61], [62], [63], [64], [65], [66], [67], [68], [69]. The goal of the defenses is to mitigate WFP in the Tor network. To mitigate WFP, the defenses mutate various aspects of the communication to render previously identified features useless. However, mitigating WFP attacks results in bandwidth and latency overheads. We consider a simple defense based on the padding of TLS records. The defense is similar in spirit to existing defense methods such as CS-BuFLO [17]. Instead of the Tor network, the defense focuses on the TLS protocol and uses only mechanisms available in the current TLS standard.

General traffic classification: Beyond WFP, statistical approaches have been used for general network traffic classification [70], [71]. These works focus on broadly identifying application classes, e.g., P2P, e-mail, or Web, using behavioral analysis [72] or detecting SSH and Skype flows [73] or Android Apps [74], both using flow-level features. Similarly, TLS features were used to identify services (e.g., Web vs.

⁴Random Forests and SVMs can also yield large models that take milliseconds to evaluate a sample and thus not suitable for WFP attacks at scale. For example, CUMUL in Fig. 1 is based on an SVM.

video) and applications (e.g., Facebook chat vs. Google Drive) [75] or Web services [76]. Unlike application or service detection, PROFi focuses on performing WFP attacks in both an open and a closed-world scenario. However, the principle behind PROFi could be applied to more general traffic classification, which we leave as future work.

Real Time Traffic Classification: FENXI [46] introduces a system that uses specialized hardware accelerators for deep learning to accelerate and enable the application of deep learning to packet processing. ILSY [77] implements four ML models in programmable data planes with the P4 language and shows the limitations of these approaches. Similarly, Xavier et al. [78] investigate how ML algorithms can be executed in the data plane. N2Net [79] and BaNaNa Split [80] discuss the implementation of binarized neural networks in the data plane. Pacheco et al. [81] review ML-based solutions to network traffic classification. In contrast to previous work, we do not rely on special hardware nor highly optimized frameworks targeting specific application scenarios only. Instead, we show how existing frameworks and commodity hardware can be used to realize WFP attacks.

IV. ATTACK MODEL AND DEFENSE

This section introduces PROFi's classifier using probabilistic models of packet sequences and the associated feature engineering. The classifier is designed for computational efficiency, i.e., relies on models with few parameters. Further, the classifier is designed for implementation as micro-services in a cloud-native deployment scenario. The design aligns with future networking concepts that feature networking on top of a MEC infrastructure, thus increasing PROFi's attack potential. The design assumes that PROFi is situated between the user and the CDN, routing is symmetric, and no VPN is used. Further, this section introduces three state-of-the-art approaches to WFP as baselines for later evaluations.

A. Overview

To operate on the flow level, PROFi learns the characteristics of the first TLS connection that is established between a user's client when retrieving a webpage. We refer to this connection as MAINFLOW. Thus, classifiers in PROFi answer the question: *Is this flow a MAINFLOW to a webpage of one of the websites that should be detected?* Specifically, PROFi does not require correlating flows from an individual client to make a prediction. Furthermore, PROFi requires only samples of MAINFLOWS from monitored websites during training, i.e., PROFi does not require samples of MAINFLOWS from unmonitored websites or flows other than MAINFLOWS. As a consequence, PROFi doesn't have to extract flows constituting a webpage retrieval from the traffic aggregate, i.e., the data PROFi requires is readily available during operation. Further, detecting webpage calls based on the initial connection can enable active interference, e.g., by dropping all traffic for a specific IP for a short time after the call is detected.

Fig. 3 illustrates the high-level operation of PROFi. The attack translates a flow's packets into a sequence of nominal

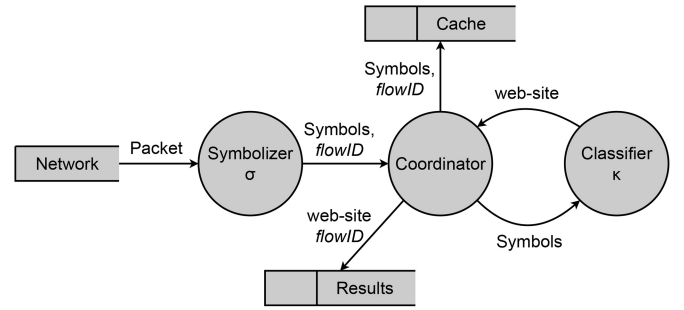


Fig. 3. Data-flow diagram of the classification procedure of PROFi. Italic script identifies meta-data.

symbols and labels the sequence with a classifier. More formally, the attack consists of three functions: The SYMBOLIZER σ , the COORDINATOR, and the CLASSIFIER κ . In addition, there are three data sources and sinks, the Network, a CACHE to store intermediate results, and persistent storage for the classification results. First, PROFi receives a packet $u \in \mathcal{U}$ from the network. The set $\mathcal{U}$ corresponds to all IP packets occurring during a TLS session. A SYMBOLIZER σ converts the packet into an ordered set of abstract symbols $\mathcal{M} \subset \mathcal{W}$, where $\mathcal{W}$ is the set of all possible symbols. $\mathcal{M}$ together with a flow identifier is passed to the COORDINATOR. The COORDINATOR accumulates $\mathcal{M}_t = \sigma(u_t)$ derived from T packets $u_1, \dots, u_T$ of the same flow. Once the T th packet of a flow arrives, the COORDINATOR passes the accumulated sequence of symbols to a CLASSIFIER κ . The CLASSIFIER determines whether the flow is a MAINFLOW from one of the monitored websites $y \in \mathcal{Y}$ and returns the result to the COORDINATOR, where it is stored.

B. Extracted Features

To classify flows, PROFi uses packet-level data from, at most, the first 30 packets, corresponding to the 10th percentile of MAINFLOW lengths from our collected data. The exact number of packets used by PROFi is subject to parameter optimization of the attack.

PROFi extracts features on the level of the packet and the level of the TLS records. For each packet, PROFi extracts the size in bytes, the TLS record types, and the direction, i.e., whether the packet travels from the client to the server or the other way around. For each TLS record in the packets PROFi extracts the TLS record length, the TLS record type, and the direction. Since existing tools such as *ssldump* do not provide all of these features, we implemented a custom traffic dissector that extracts the features from the packet capture files. PROFi does *not* use timing information between packets or TLS records. Latency, especially on the Internet, expresses high variance not caused by the visited website [82]. Instead, geographic location and difficult-to-observe conditions such as the network load explain the latency's variance [82]. Including latency would thus increase the requirements for data collection to avoid the risk of overfitting.

Fig. 4 illustrates the extracted features for two websites: *www.google.es* and *www.primevideo.com*. Negative numbers in Fig. 4 correspond to traffic sent from the

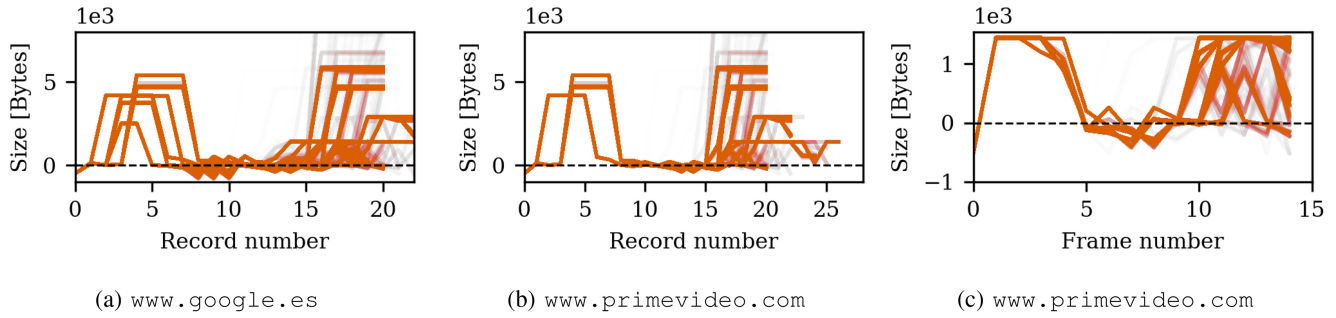


Fig. 4. Frame/TLS record sizes: negative (positive) values indicate traffic send by the client (server), resp.

client to the server. Positive numbers indicate the opposite direction. Fig. 4 shows clear differences in the constructed sequences for the websites. For example, Fig. 4(a) shows that `www.google.es` has a high variance during the TLS handshake, and the client sends larger packets to the server. In contrast, `www.primevideo.com` in Fig. 4(b) has a different handshake behavior, and the client sends fewer data to the server. Also, the order is important: for example, `www.google.es` in Fig. 4(a) has characteristic dips between records seven and ten. The sequence of `www.primevideo.com` in Fig. 4(b) has small records after the handshake until frame 15, after which the record sizes become highly variable. Moreover, websites can have a multi-modal behavior. For example, `www.google.es` in Fig. 4(a) has three TLS handshakes that vary in size and number of TLS records. The varying handshake lengths then shift the entire sequence. The remaining steps in PROFI, i.e., the SYMBOLIZER and the CLASSIFIER, have to preserve and extract these patterns.

Lastly, the sequences constructed from TLS records and frames can differ. For example, the sequence of frame sizes for `www.primevideo.com` in Fig. 4(c) differs markedly from the TLS record sizes in Fig. 4(b). TLS can package multiple records into one Transmission Control Protocol (TCP) segment. Similarly, a TLS record can span multiple TCP segments. Thus, the sequences of TLS record sizes and the frame sizes that carry the records can differ strongly. In fact, the way TLS records are packaged into TCP segments is a discriminating feature on its own [76]. In our collected data, the number of TLS records and packets is approximately equal on average but varies from website to website, confirming the potential value of this feature.

C. Symbolizer

The SYMBOLIZER converts the features extracted for each packet and record into a sequence with nominal elements that the CLASSIFIER can process. Each sequence element is constructed from three features: direction, size of record or packet, and TLS record type(s). The size of the packet or record is numerical and the direction, and TLS record types are nominal. We map the direction, size, and record types to an abstract symbol. To reduce the observation space and account for small deviations in the lengths of packets and records, we discretize the packet and record lengths.

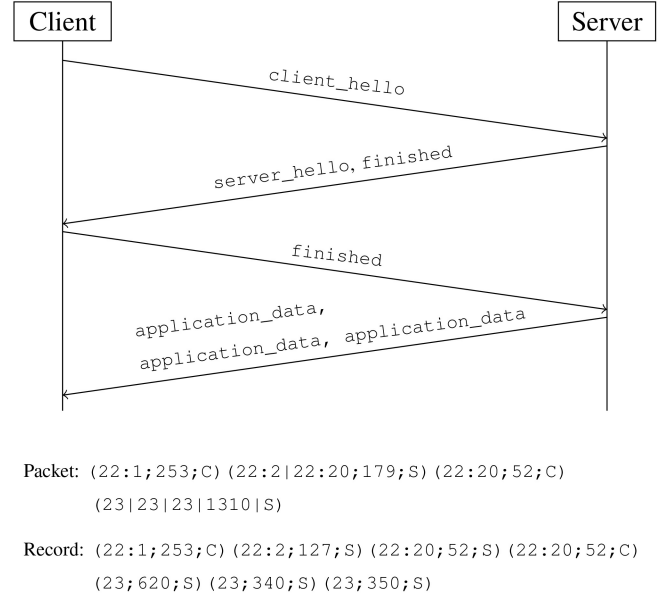


Fig. 5. Shows how the TLS 1.3 handshake and one packet of application data are mapped to a sequence of symbols. For illustration, the handshake is simplified. The sequence diagram shows the flow of packets, and labels on the arrows indicate TLS records. The resulting sequences for packet and record are below the diagram.

Formally, we define the SYMBOLIZER as a function $\sigma : \mathcal{U} \times \mathcal{T} \rightarrow 2^{\mathcal{W}}$. The set $\mathcal{T}$ corresponds to the element for which a symbol should be created, i.e., packet or record. The output is an ordered set of symbols. The order is established with the help of a function $\text{pos} : \mathcal{W} \cup \mathcal{R} \rightarrow \mathbb{N}_+$, which returns the position of the passed element inside the flow. The symbol is formed by combining the record type(s), size, and direction. To represent the record types, the SYMBOLIZER uses the corresponding decimal code from the TLS standard. For example 23 for `application_data` records, and 22 for records in the handshake, e.g., 22:1 for the `client_hello` and 22:2 for the `server_hello` [83]. Further, the SYMBOLIZER represents the direction through a letter C when the sequence element originates at the client, and S when the sequence element originates at the server.

For instance, Fig. 5 shows an example mapping the beginning of a TLS connection to a sequence of symbols. For brevity, this paragraph will elaborate on the fourth packet in Fig. 5 only. The fourth packet in Fig. 5, referred to as u , travels from the server to the client and carries three application

data records. The packet u has a size of 1310 Bytes. The records have lengths of 510, 399 and 401 Bytes. For a $\mathcal{T}$ of packet, packet u would be mapped to one symbol $\{(23|23|23;1310;S)\}$, i.e., σ returns a singleton-set. In the case of $\mathcal{T}$ of record, packet u is mapped to three symbols. One symbol per record: $\{(23;620;S), (23;340;S), (23;350;S)\}$.

For both sequence elements `packet` and `record`, symbols are categorical, i.e., symbols can be interpreted as separate characters in an alphabet. Thus, two symbols that look similar, e.g., $(23;620;C)$ and $(23;620;S)$ are different - just as two characters such as `e` and `c` are different.

The input to the CLASSIFIER is the union of the return values of σ for the first n packets of a flow $\mathcal{F} := \{u_1 \dots, u_n\}$: $O := \bigcup_{u \in \mathcal{F}} \sigma(u)$. This forms a sequence with nominal elements that are sorted based on the order in which the corresponding packets occur in the flow.

D. Classifier

The SYMBOLIZER returns sequences with nominal elements, and PROFi uses ML techniques that can operate on those. We consider two ML techniques: first-order Markov Chains (MCs) [76], and Profile Hidden Markov Models (PHMMs) [84], [85]. In addition, we consider as base-lines the k-Nearest Neighbor (kNN) classifier [86] with the Levenshtein distance [87] to measure similarity, the SVM-based CUMUL [20] attack, and the set-based IPFP [30] attack. All techniques are trained on a dataset $\mathcal{X}$ consisting of calls to multiple webpages of websites. We denote with $\mathcal{X}_{ij}$ the j th webpage of the i th website.

MC and PHMM belong to the class of Probabilistic Graphical Models (PGMs). The PGMs describe the nominal sequences, the SYMBOLIZER converts MAINFLOWS to, with the language of probability theory [37]. That is, one PGM represents MAINFLOWS of one website $y \in \mathcal{Y}$. We chose PGMs over other ML models because PGMs have few parameters, are interpretable, extensible, and signal data drift. A small number of parameters is important to make PROFi memory and computationally efficient, allowing PROFi to scale to many concurrent webpage accesses. At the same time, a small number of parameters aids the interpretability of the model, which is further improved through the specific structure of, e.g., the PHMM [84]. PGMs are readily extensible, e.g., it is straightforward to include additional variables that can improve the classification [37], e.g., the information of the CDN that is contacted. Lastly, PGMs model the data directly and thus provide a measure of how well the model fits the data. This is an important aspect for the operation of PROFi since it allows the detection of data drift, i.e., the data distribution in operation changes over time compared to the distribution of the training data. An effect that has been previously reported [51] and that we also find in our data.

PGMs do not return a specific label. Instead, PGMs compute the *likelihood*, i.e., the probability $p_y(O)$ of a sequence O given the model for website y [37]. A Maximum Likelihood Classifier (MLC) can use the likelihood for classification: An MLC returns the label corresponding to the model with the

largest likelihood, i.e., the model under which the observation has the highest probability [85]. However, MLCs are not suited for an open-world scenario. An MLC will always return a label, even if every model has a probability of zero.

To overcome this issue, we treat classification as anomaly detection. The classifier's PGMs modeling each website's MAINFLOW represent the normal behavior. If observation shows anomalous behavior, i.e., has a low probability under the PGM describing the website's behavior, the classifier rejects the observation. We define anomalous behavior based on the anomaly score $\beta_y(O)$ of a website as:

$$\beta_y(O) := \frac{\log p(y(O))}{\gamma \max_j \log p_y(\mathcal{X}_{yj})}. \quad (1)$$

Here, $p_y(O)$ corresponds to the model of website $y \in \mathcal{Y}$, and $\gamma \in \mathbb{R}_+$ is a scaling factor. The score $\beta_y(O)$ corresponds to the ratio of the log-likelihood of the model for website y to the largest log-likelihood from the training set. The free parameter γ can be used to tune the ratio. The interpretation of the ratio is as follows: If $\beta_y(O) \leq 1$, then the given sequence fits equally well to the model as the training data. If $\beta_y(O) > 1$, then the sequence is less likely than all training sequences.

In the closed-world scenario, the classifier returns the website of the model with the smallest score. This results in a multi-class classifier that returns a label $y \in \mathcal{Y}$ for every observation. In the open-world scenario, the classifier uses the score to turn each PGM into a binary classifier. If $\beta_y(O) > 1$, then the model rejects the observation O , i.e., observation O does not correspond to a MAINFLOW of website y . If $\beta_y(O) \leq 1$, then observation O is labeled as an instance of website y . This results in 3 cases: 1) All models reject sequence O , 2) one model accepts O , and 3) multiple models accept sequence O . In case one, the sample is classified as background traffic. In case two, the sample is classified as the website the model corresponds to. In case three, the classifier returns the website whose model has the smallest score.

Adjusting γ allows trading false positives for false negatives. A value smaller one reduces the risk of false positives and increases the risk of false negatives, i.e., the classifier is stricter. A value larger one increases the risk of false positives and reduces the risk of false negatives, i.e., the classifier is more lenient in its opinion of what is normal.

This approach has five advantages: 1) PGMs for websites can be trained independently from each other. Thus, models for new websites can easily be added, and existing ones can be updated without changing other models, and PROFi does not need samples for the background class, as, e.g., CUMUL does. 2) We can vary the pre-processing steps for each website independently. Varying the pre-processing steps changes the observation spaces for PGMs. Thus, the log-likelihood of PGMs cannot be directly compared since the log-likelihood is not calibrated [37]. The anomaly score solves this problem by quantifying how normal an observation behaves compared to others of the same class. 3) The likelihood of the PGMs can be used to detect changes in the data distribution and thus indicate a reduction in reliability [37]. 4) The PGMs PROFi uses are computationally efficient and can consume data in a streaming manner. That is, the symbols extracted from packets

by the SYMBOLIZER can be fed one-by-one into the models without the need to store the full sequence.

E. Defense

We implement and evaluate a defense based on the padding in the TLS record protocol, which we call Random TLS Record Size Defense (RTLSRS). RTLSRS randomly draws TLS record sizes from a uniform distribution $U(l_{min}, l_{max})$, and pads records to the desired length if necessary. After each record, RTLSRS randomly chooses between adding the next record to the data or handing the current data to the transport protocol. Thus, the sizes and sequence of packets and TLS Records change - the features that PROFI uses to detect patterns. The TLS 1.3 [83] standard allows the padding of TLS records to arbitrary sizes up to 2^{14} Bytes. Thus, the defense can readily be implemented in SSL libraries.

Characteristics that are not protected with this defense are the TLS handshake and characteristic exchanges between client and server. The handshake protocol does not support padding [83]. Thus we leave the messages as they are. Characteristic exchanges, i.e., a specific sequence of contacts between client and server, could, in principle, be obfuscated with deterministic sending of decoy packets, albeit at the expense of more bandwidth overhead. Our evaluation shows that the simple defense suffices to thwart PROFI.

Our goal with RTLSRS is to present an easy-to-implement and effective defense against PROFI. RTLSRS is not intended to outperform previous WFP defenses. In principle, existing WFP defenses should also effectively mitigate PROFI. However, existing defenses, e.g., HTTPOS [64] or CS-BuFLO [17], are often complex to implement and require continuous effort to maintain, which might hinder their adoption.

V. MODEL TRAINING

A. Data Acquisition

To evaluate the attack, we consider 96 websites located in three popular CDNs. For each CDN, we selected the top 30 websites based on the Alexa Top 1000 [88]⁵ ranking. For each website, we selected 50 random sub-pages using a javascript-enabled Web-crawler. We obtain a data set consisting of 4800 webpages. We gathered traces of every webpage daily for 70 days. We loaded every website with Chromium and the Firefox browser in headless mode. This resulted in 100 samples for each website, and a total of 9600 traces per day. In total, we collected 3 TB of traces.

Docker containers were executed on three separate physical machines running Ubuntu 20.04 and used to isolate traffic of webpage access and to maintain equal conditions for all pages. The browser is started from inside the docker container in headless mode. We traced traffic for 7 s, after which the docker container is terminated. Traffic was collected inside the docker container with the `tcpdump` utility.

The gathered data is limited in that it does not contain mobile traffic and webpages that require a previous

login. We decided against including this type of traffic since its acquisition requires tools such as Selenium, or virtualized environments that could impact the feature distribution. Websites are known to detect such technologies, potentially blocking or responding in different ways than normal [89]. Further, to log in, users normally have to access a publicly accessible landing page first. Similarly, we did not passively listen to real traffic since we did not have access to such a monitoring location.

B. Data Split and Training Procedure

We evaluate MC, PHMM, kNN, CUMUL, and IPFP in a closed-world and an open-world scenario. We divide the websites into two sets: *Foreground Sites* and *Background Sites*. The *Foreground Sites* are the sites that the classifiers should detect. The *Background Sites* are the sites that should be ignored. In the closed-world scenario, the classifiers must differentiate the pages in the *Foreground Sites*. In the open-world scenario, the classifiers must detect whether traffic belongs to a page from the *Foreground Sites*, and if so, from which one. We create the sets by randomly assigning websites to one of the two sets, such that each website type is represented in each set. For example, we ensured that both sets contain sites with adult content, news pages, etc.

The classifiers are trained on MAINFLOWS from all 70 days from the *Foreground Sites*. Flows other than MAINFLOWS are not used for training, as their characteristics differ from those of MAINFLOWS, thus hampering the ability of the PGMs to model the MAINFLOWS' characteristics. For training, we split the webpages of a website into three disjoint sets: training, validation, and test set. The training set contains 60 % of the webpages, the validation and test sets contain 20 % each. For hyper-parameter optimization, we fit the models to the training set and evaluate the performance on all 70 days in the validation set. For CUMUL, we include a background class in the open-world scenario as suggested in [20]. The background class consists of traces of one webpage from every website in the *Background Sites*. We select the parameters that resulted in the best *precision* on the validation set. That is, we prefer the models with fewer false positives. For the final results, we train the classifiers on the combined training and validation set with the selected hyper-parameters and evaluate them on all 70 days in the test set.

C. Hyper-Parameter Optimization

We use a grid-search for hyperparameter optimization. Table I lists the parameter space. The Binning Method, number of bins, number of packets, and sequence element is optimized for PHMM, MC and kNN. For kNN, we convert each of the symbols a packet is mapped to into Unicode characters and concatenate the characters to a string. As a hyperparameter for kNN, we vary the number of neighbors. In case of the PHMM, we vary its length. For CUMUL, we evaluate the hyper-parameter space of the SVM as suggested in [20]. IPFP does not have hyperparameters.

In the case of kNN, we apply the same function σ to MAINFLOW packets from all websites. That is, the same

⁵As of May 1st, the Alexa service is no longer available.

TABLE I
PARAMETER SPACE OPTIMIZED OVER

Parameter	Values
Binning Method	EQUALWIDTH, LOGARITHMIC, NONE, SINGLEBIN
Num Bins	$\{0, 1\} \cup \{10 \cdot i \mid i = 1, \dots, 10\}$
Num Packets	$\{5 \cdot i \mid i = 1, \dots, 6\}$
Sequence Element	Packet, TLSRecord
kNN num Neighbors	$\{3, 6, 9\}$
PHMM length	$\{5 \cdot i \mid i = 1, \dots, 6\}$
CUMUL c	$\{2^{11}, \dots, 2^{18}\}$
CUMUL γ	$\{2^{-3}, \dots, 2^4\}$

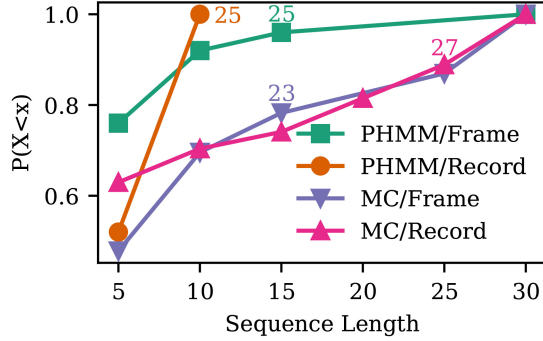


Fig. 6. CDF of the discrete sequence lengths and sequence types of PHMM and MC. Numbers in the figure give the total count of models that use the corresponding sequence type.

Sequence Element from the same number of packets from all MAINFLOWS of all websites are mapped to a symbol with the same binning method and number of bins.

In the case of MC and PHMM, we take advantage of the fact that models for websites are independent. We vary the function σ for every website. Thus, each model has its own function σ . A flow U is converted to a unique sequence for every model. With this approach, it is possible to tune the conversion to symbols in such a way that the PGMs can model them well, and at the same time, differentiate them from sequences of other websites. The parameters of the model are estimated only on the samples of the corresponding website.

D. Selected Hyperparameters

The best configuration for kNN uses 9 neighbors and the size and direction of the TLS records in the first 30 packets of the MAINFLOW. The classifier has 40 bins with record sizes of EQUALWIDTH binning. The best configuration for CUMUL is $\gamma = 2^{11}$ and $c = 2^3$.

MCs and PHMMs have diverse configurations, indicating that it is beneficial to tune the pre-processing for each website independently. Of special interest are the selected sequence lengths and sequence elements, impacting the computational cost of the attack. Longer sequences require more packets and more processing. Relying on TLS record information requires the additional parsing of TLS headers. Fig. 6 shows the Cumulative Distribution Functions (CDFs) for the best PHMM and MC models. The CDFs show the distribution over the sequence lengths and sequence elements. The numbers

in Fig. 6 indicate how many models used the respective sequence type. Fig. 6 shows that 25 PHMMs use records and 25 PHMMs use frames. The PHMMs using records require up to 10 packets only, i.e., the records contained in the first 10 packets of each MAINFLOW. PHMMs using frames require 5, 10, 15 and 30 packets. For the MC models, 23 use frames, and 27 records as sequence elements. The length of sequences varies. For both, MC and PHMM, more than 50 % of the models require only five packets. Note that the models with TLS records as sequence elements can get sequences longer than the number of packets, since one packet can carry more than one TLS record, especially during the handshake.

The fact that many MC and PHMM models rely on the first 5 packets is surprising. The first 5 packets contain mostly the TLS handshake. Depending on the TLS configuration, the size of the handshake packets and records varies. For example, servers and clients can have different extensions and exchange certificates of varying sizes [83]. Fig. 4 illustrates this, showing that even for a single website, the TLS handshake varies. The differences between the websites are enough for the PGMs to distinguish websites from each other.

E. Summary

PGMs can represent the MAINFLOWS of websites. The results show that many websites can be distinguished with as little as the first 5 to 10 packets. This is in contrast to previous work on WFP, which relies on all packets sent during the page-load. Future work on PGM-based attacks could improve the emission model and address potential multi-modal behavior in the data. Currently, the models rely heavily on the NONE binning method, particularly models for websites with dynamic content. The NONE binning method requires many training samples to get sufficient data since the packet and record sizes are not compressed, i.e., every size that occurs normally must be captured. Here, non-discrete emission models might better compress the emissions and reduce the risk of overfitting and the required samples. The mix of discrete and continuous features makes the improvement of the emission model challenging. Similarly, our data shows that webpages can express multi-modal behavior. This poses a challenge to the used PGMs, since their ability to capture multi-modal sequences is limited. Investigating the performance of mixture models, e.g., a mixture of MC and PHMMs to model one website could be an interesting aspect for future work.

We believe that the attack can be thwarted by randomizing or standardizing packet and record sizes due to the reliance of the models on packet and record sizes since SINGLEBIN reduces the average precision to only 20 % on the validation set. Performance decreases because SINGLEBIN removes the packet and record size information, leaving only direction and TLS record type as information. Randomizing packet and record sizes has a similar effect since randomizing essentially removes size information from the data. Furthermore, the large number of models that work well with only five packets indicate that the TLS handshake is important for classification. Standardizing the TLS handshake, i.e., handshakes

use the same sequence and size of packets independent of the concrete TLS settings, could help mitigate WFP as well.

VI. EVALUATION

The evaluation focuses on the metrics precision $:= \frac{TP}{TP+FP}$ and recall $:= \frac{TP}{TP+FN}$, where TP are the true positives, FP the false positives, and FN the false negatives [86]. High precision means reliability, i.e., retrieved instances usually belong to the predicted class. High recall means that the classifier retrieves most of a class's samples. Usually, there is an interdependence between precision and recall, i.e., improving a classifier's recall comes at the cost of reduced precision, and vice versa. PROFi's classifier can trade precision for recall by adjusting the parameter γ appropriately. We do not discuss the accuracy of the classifiers since the accuracy has little expressiveness. With almost 100 classes, a classifier can achieve 99 % accuracy by predicting one class for all samples [86], analogous to classification problems with imbalanced classes. Here, precision and recall better reflect a classifier's performance.

We evaluate PROFi's classifier in an open- and closed-world scenario, with and without defense, and under asymmetric routing. Further, we compare PROFi's classifier against three baselines. We evaluate RTLSRS by sampling record sizes from a uniform distribution with a lower limit of 100 and an upper limit in the set $\{100i + 100 \text{ Bytes} \mid i = 0, \dots, 16\}$.

With asymmetric routing, packets traveling from server to client can take a different route than packets from client to server [90]. Thus, PROFi might not observe both directions. Since asymmetric routing frequently occurs in Wide Area Networks [90], evaluating PROFi under its influence is important.

To relate PROFi's performance to that of previous attacks, we evaluate three baselines: kNN, CUMUL [20], and IPFP [30]. kNN has been used in previous attacks [16], [19], [21] and operates on the same feature space as PROFi, i.e., on the sequence of packets and TLS frames. kNN stores the training data verbatim and is thus a strong baseline [86]. CUMUL [20] is one of the most cited WFP attacks and uses Support Vector Machines (SVMs) and traffic features from the full page load. IPFP [30] is a recent attack leveraging the IP addresses that occur during the page-loads to form fingerprints.

kNN, CUMUL, and IPFP are not directly applicable in the attack scenario in Section IV. kNN has a large memory overhead and is costly to evaluate even with appropriate data structures. CUMUL and IPFP require access to all flows of a page load, making them hard to use with traffic aggregates. Especially IPFP relies heavily on correctly extracting page loads from traffic aggregates, a mostly open problem. In contrast to kNN, CUMUL's SVM is not suited to model variable-length sequences with nominal elements.

A. Closed World vs. Open World

Fig. 7(c) shows violinplots for precision and recall for the open- and closed-world scenarios with and without defense. Asymmetric routing is indicated with the endings *fc2s* and *fs2c*. For *fc2s*, the traffic from the client to the server is

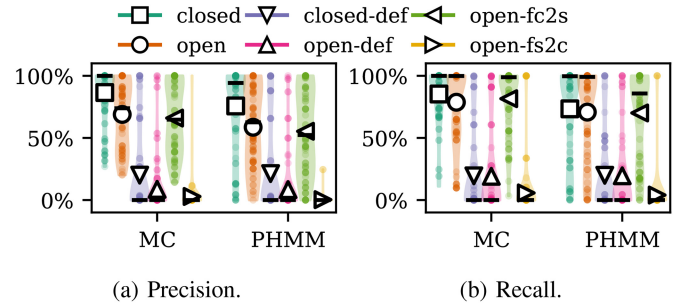


Fig. 7. Precision and recall for the closed-, and open world scenario, with defense (-def) scenarios, filtered traffic from server to client (-fs2c), and from client to server (-fc2s).

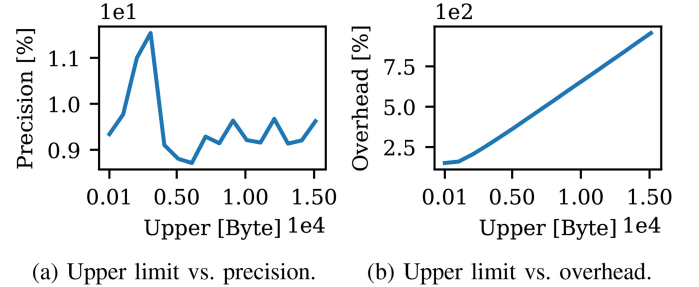


Fig. 8. Line plot showing precision and overhead as a function of RTLSRS' upper sizes.

unobserved. For *fs2c*, the traffic from the server to the client is unobserved. Markers indicate the average, horizontal bars the median, and small dots are actual samples. Each sample corresponds to the average for one website in the *Foreground Sites* across all days. For example, a sample with 95 % recall means that the model missed 5 % of the webpages belonging to the website across all days.

MC and PHMM achieve better precision and recall in the closed-world scenario, which is expected. In the closed-world scenario, MC achieves an average precision of 86.5 % and an average recall of 85.4 %, and the PHMM of 75.7 %, and 73.4 %, respectively. In the open-world scenario, the precision and recall reduce to 68.9 % and 78.7 % for MC, and 58.6 % and 70.8 % for PHMM. However, the distribution of precision and recall is skewed, as Fig. 7(c) shows. Both models achieve a median precision and recall of > 99 % in the closed-world scenario. The median precision for MC in the closed-world scenario is 99.9 %, and 94.2 % for the PHMM. In the open-world scenario, the median precision drops for both models close to the average. This indicates that the PGMs confuse *Background Sites* with *Foreground Sites*. Still, both models have a precision of > 90 % for one-third of the *Foreground Sites* in the open-world scenario.

B. RTLSRS Defense

Fig. 8(c) shows the precision and the overhead as a function of the uniform distribution's upper size. Fig. 8 shows that the precision is low for all upper limits, ranging between 8 % and 12 %, with a minimum of 8.71 % for an upper size of 6100 Byte. In contrast, Fig. 8(b) shows that the overhead increases linearly with the upper size. An upper size of

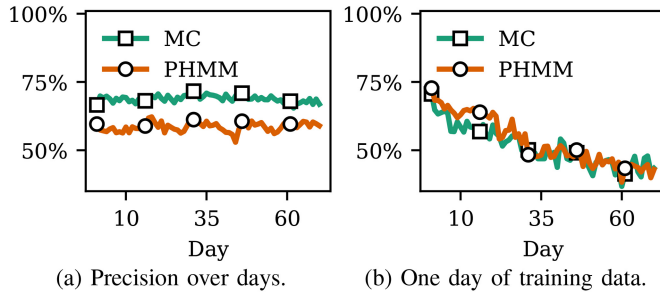


Fig. 9. Development of precision over multiple days.

100Bytes has the smallest overhead with 150 %. The upper size of 6100 has an overhead of 425 %. We thus investigate the RTLSRS' effect on the classification performance for an upper size of 100Bytes. This reduces the average precision to 9.35 % while keeping overhead low.

Fig. 7(c) shows that the RTLSRS defense from Section IV-E effectively mitigates PROFi. The RTLSRS defense reduces the precision and recall of both models to $\sim 20\%$ in the closed-world scenario. In the open-world scenario, RTLSRS reduces the precision of both models to less than 10 %. The recall is less affected and reduces to $\sim 19\%$.

The RTLSRS defense thus has two effects: The models miss many webpages (low recall), and the credibility of the retrieved webpages is small (low precision). On average, more than 90 % of the webpages the models classify as belonging to a specific website belong to another. Still, even in the presence of defense, a few websites exist for which PROFi achieves high precision and recall, as Fig. 7(c) shows.

C. Asymmetric Routing

Fig. 7(c) shows that the effect of asymmetric routing on PROFi's performance depends heavily on the direction of unobserved traffic. If traffic from the server to the client remains unobserved, the average precision and recall of MC and PHMM deteriorate to less than 6 %.

If traffic from the client to the server remains unobserved, then the average precision of MC and PHMM reduces by $\sim 3\%$. Similarly, the distribution of the recall for the PHMM shifts towards lower values, as Fig. 7(c) shows. The median recall reduces by $>13\%$, while the average reduces by 0.7 %. For the MC, the average recall improves from 78.7 % to 81.6 %, while the median stays high at 98.9 %.

This outcome is reasonable. Few packets travel from client to server, amounting to TLS messages and HTTP commands with small variance, making websites hard to differentiate. Most packets travel from server to client and have a high variance, as Fig. 4 shows. Missing traffic from the server to the client deprives the PGMs of much of the information they use for classification.

D. Training Data Evaluation

Fig. 9(a) shows the average precision over days when training on all training data. Fig. 9(a) shows that the precision of all models remains approximately constant over time. The recall scores show similar behavior.

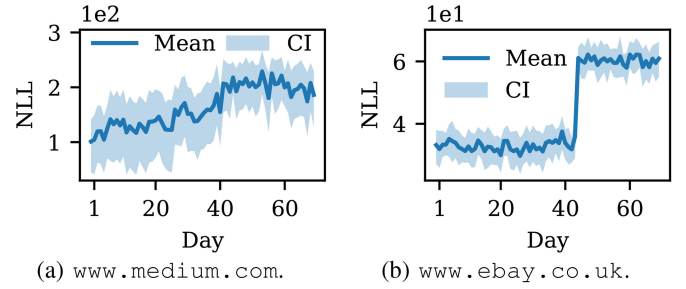


Fig. 10. Development of the NLL for the MC trained on one day. The shaded area is the bootstrapped 99 % CI of the mean.

Fig. 9(b) shows the average precision per day when training on traces of the first day. Fig. 9(b) shows that the precision decreases over time. The average precision of MC and PHMM drops to 66.2 % and 42.5 %. This behavior is not unexpected and has previously been observed [51]. This indicates that attackers must continuously gather new data to update the deployed models for websites with dynamic content.

Fig. 10 illustrates this and shows the average Negative Log Likelihood (NLL)⁶ and its 99 % Confidence Interval (CI) for the MCs fitted to data from www.medium.com (Fig. 10(a)), and www.ebay.co.uk (Fig. 10(b)). Fig. 10 shows two examples of distributional drift that can occur in an operative system: A slow and steady drift and a sudden change.

Fig. 10(a) shows that the average NLL has an increasing trend indicating that the input diverges from the data the model was fitted with. To continuously operate the system, the adversary thus has to continuously update its training data and model to track the drift in the input's data distribution.

In contrast, Fig. 10(b) shows a sudden jump after 42 days. This indicates a persistent change in the input data's distribution, making the model unreliable. The change is caused by a different length of the server's TLS finished record. If such a change occurs, the adversary must collect new samples to adjust the model to the new input distribution.

The advantage of PGMs is that they readily detect drift in the input's distribution. Further, PGMs allow the investigation of what changed [37], [76]. Here, the PGMs allow the identification of the transitions with low probability, directly pinpointing the change in the exchanged messages.

E. Comparison to Baselines

Table II shows the average precision and recall for MC and PHMM, and the three baselines kNN, CUMUL, and IPFP in the closed-, and open-world scenario. Table II shows that PROFi is competitive to state-of-the-art models. In the closed-world scenario, kNN is the best model with an average precision of 91.8 % and an average recall of 91.44 %. kNN achieves this at the expense of increased memory and computational requirements, whereas PROFi relies on computationally cheap ML models with few parameters. Fig. 1 illustrates this fact and shows the number of pages per second classifiers classify in an offline setting. Fig. 1 shows that kNN achieves short of 10 classifications per second,

⁶The NLL is uncalibrated and cannot be compared between models.

TABLE II
PRECISION, RECALL AND ACCURACY FOR PHMM, MC, AND THE
BASELINES IN THE OPEN AND CLOSED SCENARIO

		PHMM	MC	KNN	CUMUL	IPFP
Closed	Precision	75.74	86.51	91.80	63.43	83.68
	Recall	73.40	85.35	91.44	62.18	85.17
Open	Precision	58.58	68.90	61.89	30.74	80.60
	Recall	70.76	78.71	88.71	60.97	85.19

whereas MC and PHMM achieve between 30 000 and 100 000 classifications per second.⁷

In the open-world scenario, IPFP is the best model with a precision of 80.6% and a recall of 85.2%. In contrast to PROFi, IPFP requires additional pre-processing steps to extract a user's page-load from the traffic aggregate. The authors do not provide a solution to this problem. Detecting for one user whether the user loads more than one page and detecting the onset of the second page load is already a challenging task [19], [22], [29], [36]. Extracting one page load from traffic aggregates with high precision is much more complex and will have limited accuracy in practice.

F. Summary

Our results show that PGMs classify traffic based on the first 5 to 30 packets of the first initiated TLS session when retrieving a webpage and are competitive to state-of-the-art attacks. This simplifies WFP attacks since page loads do not have to be retrieved from a mix of traffic. Furthermore, our evaluation shows that the attack works with only the traffic from the server to the client, i.e., can handle asymmetric routing. In addition, the computational and memory complexity of the PGMs is low and could allow adversaries to perform WFP in real-time and at scale. Our results show that the PGMs have a higher recall than precision. In scenarios where the precision is of interest, the attacker can trade recall for precision by setting the classifier's parameter γ to a value smaller one. A value smaller one requires a log-likelihood at inference time that is smaller than the largest log-likelihood at training time, i.e., for which the learned model fits better than the worst model during training. Fortunately, the RTLSRS defense that SSL libraries can readily implement thwarts the attack at the cost of a 150% increase in bandwidth.

Finally, we note that an adversary could easily improve PROFi by conditioning the probability of a website on additional variables such as the visited CDN, the time of the day, or other correlated variables. For example, calculating the probability for a website y could then be $p(y | O)p(y | c)p(y | t)$, where c identifies the CDN the website is located on and t the time of the day. For example, conditioning on the CDN would essentially reduce the hypothesis space to just the websites in

the CDN. Extending the classifier with this information could be an interesting avenue for future work.

VII. PROTOTYPE DESIGN & RESULTS

This section presents a micro-service-based prototype that implements the PROFi attack. The prototype is based on readily available open-source software and Commodity of the Shelf (COTS) hardware to showcase the potential threat PROFi poses and give an impression of the scale WFP attacks might operate at, e.g., by operating on MEC infrastructure. After establishing the ML-related performance in the previous section, this section's evaluation focuses on networking-related performance indicators.

A. Design & Architecture

1) *Design: NFV and COTS.* We believe that the principle of NFV is a good fit for PROFi. NFV can realize each PGM as one Virtual Network Function (VNF). NFV enables the scaling of ProFi across multiple servers, horizontal scaling of expensive PGMs, and the deployment of new, or updating of existing PGMs at runtime. This makes PROFi amenable to the deployment on top of MEC infrastructure. The low computational cost of the PGMs alleviates the need for special hardware accelerators and lowers the burden of deploying ProFi in practice.

Middlebox Design. A middlebox allows the attacker to interfere with a victim's traffic. The prototype could drop the remaining packets of a flagged flow. The alternative, i.e., mirroring traffic to an analysis server, does not easily allow this type of interference since switches start to sample traffic at high rates, which will result in incorrect classifications.

2) *Architecture:* We implement the prototype on top of the OpenNetVM platform [92]. OpenNetVM is a high-performance, zero-copy NF platform supporting containerization that simplifies the development of VNFs. The prototype implements the classification procedure from Fig. 3 using a micro-service-based architecture. Fig. 11 shows the structure of the prototype. We map the procedure in Fig. 3 into five services, i.e., VNFs: The TLSFilter, TLSRecDet, Symbolizer, PGM, and Coordinator.

The TLSFilter extracts TLS traffic and forwards it to the TLSRecDet. The TLSRecDet extracts TLS Records from the traffic and passes this information to all Symbolizers. Each Symbolizer converts the extracted meta-data into a discrete symbol and forwards the symbol to one PGM. The PGM processes the symbol and, once the PGM processed a configured amount of symbols for a flow, forwards an anomaly score to the Coordinator. The Coordinator labels a flow based on the received anomaly scores.

VNFs communicate with each other using User Datagram Protocol (UDP) packets. This makes the deployment of PROFi to a cluster simple. The micro-service-based architecture enables the scaling, addition, and updating of models with new parameters. For example, additional PGMs or data preparations can easily be added by starting a new service. Already running services can be scaled as needed by deploying new instances.

⁷Data structures to improve the inference time, e.g., KDTrees, are not applicable since they require real-valued inputs [91]. Our input consists of sequences with nominal elements. Further, experiments using the packet size of the first 30 packets as input features showed that the KDTree reduced the inference time not as strongly as expected, i.e., only by a factor of three compared to a linear scan. This is not unexpected [91].

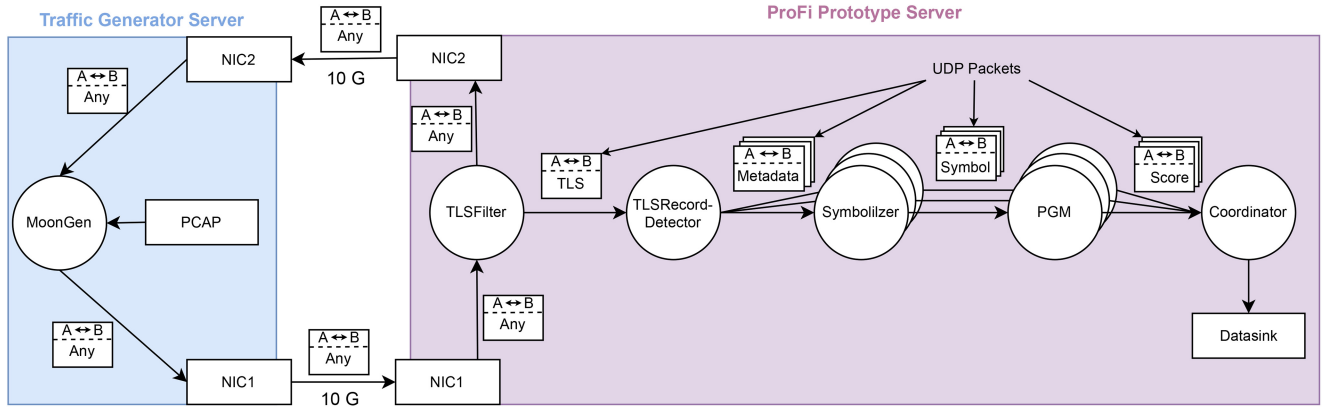


Fig. 11. Testbed setup and prototype structure. Circles correspond to processes, i.e., VNFs. Rectangles to data sources and sinks. The prototype is implemented on top of OpenNetVM. VNFs in OpenNetVM exchange information via UDP packets.

TABLE III
TRACE STATISTICS

Trace	Websites/s	TCP Flows/s	Packets/s [Mpps]
high-pps	147.71	46 551.04	2.24
high-fps	153.15	47 975.59	1.95
regular	424.17	16 446.72	1.65

B. Prototype Results

1) *Material and Methods*: The prototype operates on a server with an Intel Xeon CPU E5-265 with 24 cores, 126 GB RAM, and an Intel X540-AT2 network card with two 10 Gbit/s ports (Fig. 11). The server is connected back-to-back to a second server with identical hardware running MoonGen [93]. MoonGen replays PCAPs constructed from the traces we collected according to Section V-A, i.e., not with passively collected traces. MoonGen replays the PCAPs as fast as possible, i.e., at the full 10 Gbit/s. The prototype can use 17 out of the 24 cores for PGMs. OpenNetVM blocks four cores, and three cores are allocated to the TLSFilter, TLSRecDet, and Coordinator.

We create two challenging workloads and one regular workload to benchmark the prototype. Table III lists the workload characteristics. The regular workload consists of a random sample of webpage calls. The adversarial workloads *high-pps*, and *high-fps* challenge the prototype with a higher packet and flow rate than the regular workload. This challenges the TLSFilter, TLSRecDet, and PGM services, respectively. Each trace has a volume of around 13 GB, resulting in a 10 s measurement on 10 G.

2) *Number of Classifiable Websites*: Fig. 12(c) shows the number of PGM services that can be co-located on one CPU core. We evaluate a small and large MC and PHMM model. The big PHMM model has 639 parameters. The small PHMM has 114 parameters. The MC models have 11 078 and 5 parameters respectively. The big model require 30 packets, the small models 5 packets for classification. For each PGM and workload, we vary the number of PGMs in $\{1, \dots, 10\}$. We report the highest number of co-located PGMs that could handle the traffic.

Fig. 12(c) shows that between 1 and 8 PHMMs can be co-located on one CPU core. The values are similar across workloads. Fig. 12(c) shows that between 6 and 8 MCs can be co-located on one CPU core. A CPU core can fit more MCs than PHMMs, which is expected since the PHMM requires more computational resources to compute the log-likelihood of a sequence. Depending on the models' sizes, the prototype could run between 20 and 100 PGM services.

3) *Impact on User Traffic*: Dataplane traffic traverses only the TLSFilter service. Fig. 12(a) illustrates the average processing times of the TLSFilter as violinplot. Each violin is computed from 20 runs. Fig. 12(a) shows that the TLSFilter is cheap and takes on average between $0.2\mu\text{s}$ and $0.33\mu\text{s}$ for each packet. Fig. 12(a) shows that the number of packets needed for classification affects the processing time of the TLSFilter function. This is expected since more packets result in a larger state of the TLSFilter function, which increases the processing time.

4) *Time-to-Label (TTL)*: The TTL is the duration between the last required packet entering the prototype and when the label is available [46]. For example, if 30 packets are required to classify a flow, then the TTL is the difference between the arrival time of the 30th packet and the time the Coordinator labels the flow. We do not use the arrival time of the first packet because this would introduce noise. The TTL would not only depend on the latency of the prototype but also on the inter-arrival time of packets in the trace.

We evaluate the TTL for the small and big PGM variants. We measure the TTL with one PGM each. Fig. 12(b) shows violin plots of the TTL. Fig. 12(b) shows a large variance in the TTL, ranging from 0.001 ms up to 1 000 ms. The median TTL for small models is at ~ 0.01 ms for PHMM and MC. The median values for the big PGMs vary. The big MC has a median TTL of ~ 100 ms, and the PHMM of ~ 0.06 ms. However, the distribution of TTLs for the PHMM is heavy tailed, resulting in an average of ~ 200 ms.

C. Summary

We implemented a prototype that classifies traffic with PGMs and evaluated it with real traffic traces. The prototype can hold 10 Gbits, corresponding to up to 424 Website

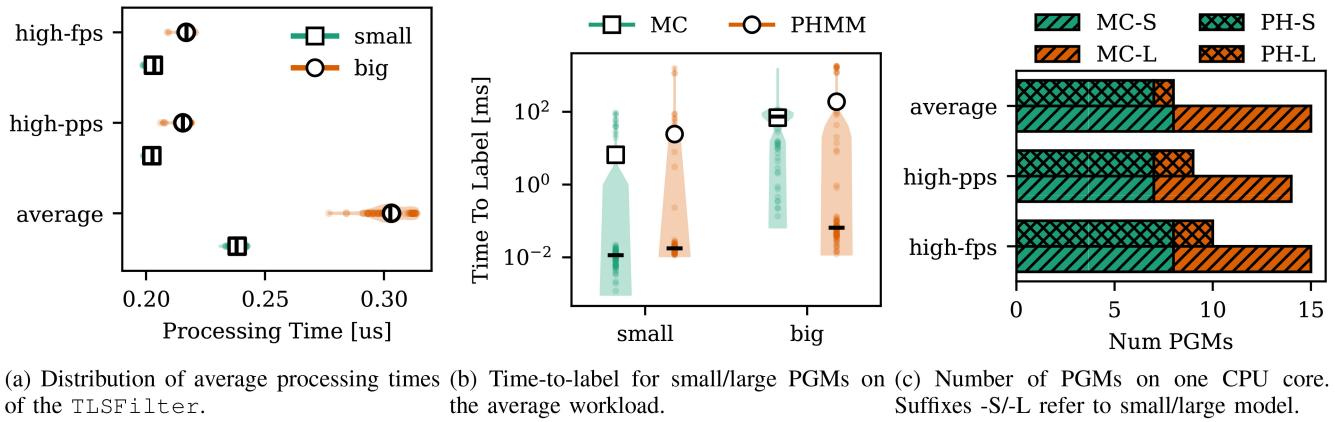


Fig. 12. (a) Distribution of average processing times of the *TLSFilter*. (b) Time-to-label for small/large PGMs on the average workload. (c) Number of PGMs on one CPU core. Suffixes -S/-L refer to small/large model.

calls per second. Statistical traffic classification with PGMs is thus feasible in the network at line rate. Our evaluations show that the prototype can monitor up to 100 websites. The results of this section have an impact beyond the WFP use-case since the *PROFi* could also label traffic for legitimate reasons.

At the same time, our work opens up new research opportunities in the area of traffic classification. We present the first design of a micro-service-based system able to perform WFP at the network edge. The investigation of how such systems, e.g., for statistical traffic classification, might be operated on top of a MEC infrastructure with computing facilities at different distances to the user could be an interesting avenue of future work.

VIII. ETHICAL CONSIDERATIONS

We carefully take steps to ensure that all captured data is in compliance with ethical standards. First, no personal data was collected during crawls of public and popular websites. Second, we limit the crawling rate to ensure that the websites are not impacted. We limit crawls to 100 per day for websites within the Alexa Top 1000 Ranking to minimize the impact of our study.

PROFi demonstrates the ability of an adversary to launch WFP attacks at scale. This raises an ethical question, as censors can use published attacks to violate user privacy. To mitigate this effect, we present a defense that can readily be implemented alongside the attack. In addition, we believe that the preemptive exposure of potential vulnerabilities is necessary to develop effective and efficient countermeasures. With our results we show the practicability and severity of WFP, sending an impulse to discuss the implementation of defense mechanisms not only in the Tor network, but also in common SSL libraries, and the TLS standard.

IX. CONCLUSION

Despite the success of the encrypt-everything movement that resulted in a substantial deployment of TLS, Internet users are still at risk of pervasive monitoring attacks. To show the feasibility of pervasive monitoring of Internet traffic at

scale, we design and evaluate *PROFi*, a Website Fingerprinting (WFP) attack on the flow level that de-anonymizes encrypted network traffic of websites that Internet users visit. To scale to large traffic aggregates, *PROFi* uses computationally cheap Machine Learning (ML) models and requires only the direction, packet size, and TLS record types of, at most, the first 30 packets of a flow. *PROFi* achieves a precision and recall of 86.51 % and 85.53 % in a closed-world, and 68.90 % and 78.71 % in an open-world scenario with 96 websites and 4800 webpages. To show that *PROFi* could be deployed by ISPs or other censoring entities at scale, we show that *PROFi* can operate as a middlebox in an online scenario and evaluate a prototype implementation in a testbed with real traffic traces. We show that *PROFi* has the potential to process up to 424 websites/s at 10G—a scale that is required to run the attack on the Internet, yet a scale to which current WFP attacks are not applicable. Further, the prototype shows that *PROFi* can label traffic within one millisecond after receiving the last required packet, which could allow active interference with users' traffic. The results suggest that WFP can be performed at scale and threatens user privacy. Yet, we show that the attack can be mitigated by standard features available in TLS 1.3. To improve Internet privacy, we posit that such defense mechanisms should find their way into TLS libraries to protect user privacy on the Internet. Further, our analysis shows that standardizing the TLS handshake could help to improve privacy since *PROFi* frequently relied on characteristics of the handshake for classification.

Future work on PGM-based WFP attacks could investigate the inclusion of auxiliary data into calculating the probability of a website to improve the classification performance. In addition, attacks using tree-based learners could be evaluated. Tree-based models can potentially operate in the match-action pipeline of switches [77] if small enough. We believe the necessary feature engineering and network integration to be challenging, making this evaluation an interesting aspect for future work. Further, the operation of the proposed microservice-based implementation on top of MEC infrastructure could be an interesting avenue for future work in statistical traffic classification.

REFERENCES

- [1] R. Barnes et al., "Confidentiality in the face of pervasive surveillance: A threat model and problem statement," IETF, RFC 7624, Aug. 2015. [Online]. Available: <https://www.rfc-editor.org/info/rfc7624>
- [2] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. E. Hoffman, "Specification for DNS over transport layer security (TLS)," IETF, RFC 7858, May 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc7858>
- [3] P. E. Hoffman and P. McManus, "DNS queries over HTTPS (DoH)," IETF, RFC 8484, Oct. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8484>
- [4] C. Huitema and E. Rescorla, "Issues and requirements for server name identification (SNI) encryption in TLS," IETF, RFC 8744, Jul. 2020. [Online]. Available: <https://www.rfc-editor.org/info/rfc8744>
- [5] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood (Internet Eng. Task Force, Fremont, CA, USA). *TLS Encrypted Client Hello*. (Aug. 2021). [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-ietf-tls-esni-13>
- [6] S. Friedl, A. Popov, A. Langley, and S. Emile, "Transport layer security (TLS) application-layer protocol negotiation extension," IETF, RFC 7301, Jul. 2014. [Online]. Available: <https://www.rfc-editor.org/info/rfc7301>
- [7] Z. Chai, A. Ghafari, and A. Houmansadr, "On the importance of encrypted-SNI (ESNI) to censorship circumvention," in *Proc. FOCI*, Aug. 2019, p. 19.
- [8] K. Bock, L.-H. Merino, D. Fifield, A. Houmansadr, and D. Levin, "Exposing and circumventing China's censorship of ESNI," Jul. 2020. [Online]. Available: https://gfw.report/blog/gfw_esni_blocking/en/
- [9] M. Liberatore and B. N. Levine, "Inferring the source of encrypted HTTP connections," in *Proc. ACM CCS*, 2006, pp. 255–263.
- [10] S. E. Coull, M. P. Collins, C. V. Wright, F. Monrose, and M. K. Reiter, "On Web browsing privacy in anonymized NetFlows," in *Proc. SS*, Boston, MA, USA, 2007, pp. 1–14.
- [11] D. Herrmann, R. Wendolsky, and H. Federrath, "Website fingerprinting: Attacking popular privacy enhancing technologies with the multinomial Naïve-Bayes classifier," in *Proc. ACM CCSW*, Chicago, IL, USA, 2009, pp. 31–42.
- [12] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proc. ACM WPES*, Chicago, IL, USA, 2011, pp. 103–114.
- [13] X. Cai, X. C. Zhang, B. Joshi, and R. Johnson, "Touching from a distance: Website fingerprinting attacks and defenses," in *Proc. ACM CCS*, Raleigh, NC, USA, 2012, pp. 605–616.
- [14] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE SSP*, San Francisco, CA, USA, 2012, pp. 332–346.
- [15] T. Wang and I. Goldberg, "Improved website fingerprinting on Tor," in *Proc. ACM WPES*, Berlin, Germany, 2013, pp. 201–212.
- [16] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *Proc. SEC*, San Diego, CA, USA, 2014, pp. 143–157.
- [17] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. CCS*, Scottsdale, AZ, USA, 2014, pp. 227–238.
- [18] X. Gu, M. Yang, and J. Luo, "A novel website fingerprinting attack against multi-tab browsing behavior," in *Proc. IEEE CSCWD*, Calabria, Italy, 2015, pp. 234–239.
- [19] T. Wang and I. Goldberg, "On realistically attacking Tor with website fingerprinting," *Proc. Privacy Enhanc. Technol.*, vol. 2016, no. 4, pp. 21–36, 2016. [Online]. Available: <https://petsymposium.org/popets/2016/>
- [20] A. Panchenko et al., "Website fingerprinting at Internet scale," in *Proc. NDSS*, San Diego, CA, USA, 2016, p. 6.
- [21] J. Hayes and G. Danezis, "K-fingerprinting: A robust scalable website fingerprinting technique," in *Proc. SEC*, 2016, pp. 1187–1203.
- [22] Y. Xu, T. Wang, Q. Li, Q. Gong, Y. Chen, and Y. Jiang, "A multi-tab website fingerprinting attack," in *Proc. ACM ACSAC*, 2018, pp. 327–341.
- [23] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM CCS*, 2018, pp. 1928–1943.
- [24] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, "Triplet fingerprinting: More practical and portable website fingerprinting with N-shot learning," in *Proc. ACM CCS*, London, U.K., 2019, pp. 1131–1148.
- [25] V. Rimmer, D. Preuveneers, M. Juárez, T. v. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. NDSS*, San Diego, CA, USA, 2018, pp. 1–8.
- [26] Z. Zhuo, Y. Zhang, Z.-L. Zhang, X. Zhang, and J. Zhang, "Website fingerprinting attack on anonymity networks based on profile hidden Markov model," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 5, pp. 1081–1095, May 2018.
- [27] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient Website fingerprinting attack based on deep learning," *Proc. Privacy Enhanc. Technol.*, vol. 2019, no. 4, pp. 292–310, 2019. [Online]. Available: <https://petsymposium.org/popets/2019/popets-2019-0070.php>
- [28] M. Shen, Y. Liu, S. Chen, L. Zhu, and Y. Zhang, "Webpage fingerprinting using only packet length information," in *Proc. IEEE ICC*, Shanghai, China, 2019, pp. 1–6.
- [29] W. Cui, T. Chen, and E. Chan-Tin, "More realistic website fingerprinting using deep learning," in *Proc. IEEE ICDCS*, Singapore, 2020, pp. 333–343.
- [30] N. P. Hoang, A. A. Niaki, P. Gill, and M. Polychronakis, "Domain name encryption is not enough: Privacy leakage via IP-based website fingerprinting," *Proc. Privacy Enhanc. Technol.*, vol. 2021, no. 4, pp. 420–440, 2021. [Online]. Available: <https://petsymposium.org/popets/2021/popets-2021-0078.php>
- [31] T. Pulls and R. Dahlberg, "Website fingerprinting with website oracles," *Proc. Privacy Enhanc. Technol.*, vol. 2020, no. 1, pp. 235–255, 2020. [Online]. Available: <https://petsymposium.org/popets/2020/popets-2020-0013.php>
- [32] T. Wang, "High precision open-world website fingerprinting," in *Proc. IEEE SP*, San Francisco, CA, USA, 2020, pp. 152–167.
- [33] Q. Sun, D. R. Simon, Y.-M. Wang, W. Russell, V. N. Padmanabhan, and L. Qiu, "Statistical identification of encrypted Web browsing traffic," in *Proc. S&P*, 2002, pp. 19–30.
- [34] K. Abe and S. Goto, "Fingerprinting attack on Tor anonymity using deep learning," in *Proc. APAN*, Hong Kong, 2016, pp. 1–6.
- [35] M. Trevisan, F. Soro, M. Mellia, I. Drago, and R. Morla, "Does domain name encryption increase users' privacy?" *ACM SIGCOMM Comput. Commun. Rev.*, vol. 50, no. 3, pp. 16–22, Jul. 2020.
- [36] W. Cui, T. Chen, C. Fields, J. Chen, A. Sierra, and E. Chan-Tin, "Revisiting assumptions for website fingerprinting attacks," in *Proc. ACM AsiaCCS*, Auckland, New Zealand, 2019, pp. 328–339.
- [37] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques—Adaptive Computation and Machine Learning*. Cambridge, MA, USA: MIT Press, 2009.
- [38] S. Studer et al., "Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology," *Mach. Learn. Knowl.*, vol. 3, no. 2, pp. 392–413, 2021.
- [39] ETSI. "Multi-access edge computing (MEC)." 2021. [Online]. Available: <https://www.etsi.org/technologies/multi-access-edge-computing>
- [40] C. Labovitz, S. Iekel-Johnson, D. McPherson, J. Oberheide, and F. Jahanian, "Internet inter-domain traffic," in *Proc. ACM SIGCOMM*, New Delhi, India, 2010, pp. 75–86.
- [41] B. Ager, W. Mühlbauer, G. Smaragdakis, and S. Uhlig, "Web content cartography," in *Proc. IMC*, Berlin, Germany, 2011, pp. 585–600.
- [42] T. Böttger, F. Cuadrado, and S. Uhlig, "Looking for hypergiants in PeeringDB," *ACM Comput. Commun. Rev.*, vol. 48, no. 3, pp. 13–19, 2018.
- [43] D. Fifield, C. Lan, R. Hynes, P. Wegmann, and V. Paxson, "Blocking-resistant communication through domain fronting," *Proc. Priv. Enhancing Technol.*, vol. 2015, no. 2, pp. 46–64, Jun. 2015. [Online]. Available: <https://petsymposium.org/popets/2015/popets-2015-0009.php>
- [44] P. Jauniškis. "VPN statistics: Users, markets, & legality." Accessed: Mar. 2022. [Online]. Available: <https://surfshark.com/blog/vpn-users>
- [45] K. Ismailaj, M. Camelo, and S. Latré, "When deep learning may not be the right tool for traffic classification," in *Proc. IEEE IM*, Bordeaux, France, May 2021, pp. 884–889.
- [46] M. Gallo, A. Finamore, G. Simon, and D. Rossi, "FENXI Fast in-network analytics," in *Proc. ACM SEC*, 2021, p. 1–8.
- [47] R. Geirhos et al., "Shortcut learning in deep neural networks," *Nat. Mach. Intell.*, vol. 2, no. 11, pp. 665–673, Nov. 2020.
- [48] K. P. Murphy, *Machine Learning: A Probabilistic Perspective* (Adaptive Computation and Machine Learning Series). Cambridge, MA, USA: MIT Press, 2012.
- [49] W. Aqeel, B. Chandrasekaran, A. Feldmann, and B. M. Maggs, "On landing and internal Web pages: The strange case of Jekyll and hyde in Web performance measurement," in *Proc. ACM IMC*, 2020, pp. 680–695.
- [50] A. Hintz, "Fingerprinting websites using traffic analysis," in *Proc. PET*, vol. 2482, 2002, pp. 171–178. [Online]. Available: https://link.springer.com/chapter/10.1007/3-540-36467-6_13

- [51] M. Juárez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt, "A critical evaluation of website fingerprinting attacks," in *Proc. ACM CCS*, 2014, pp. 263–274.
- [52] X. Gong, N. Kiyavash, and N. Borisov, "Fingerprinting websites using remote traffic analysis," in *Proc. ACM CCS*, Chicago, IL, USA, 2010, pp. 684–686. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-45744-4_2
- [53] M. Juárez, M. Imani, M. Perry, C. Díaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. ESORICS*, vol. 9878, 2016, pp. 27–46.
- [54] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *Proc. USENIX Security*, Aug. 2017, pp. 1375–1390.
- [55] D. Lu, S. Bhat, A. Kwon, and S. Devadas, "DynaFlow: An efficient website fingerprinting defense based on dynamically-adjusting flows," in *Proc. ACM WPES*, 2018, pp. 109–113.
- [56] W. De la Cadena et al., "TrafficSliver: Fighting website fingerprinting attacks with traffic splitting," in *Proc. CCS*, 2020, pp. 1971–1985.
- [57] R. Meier, V. Lenders, and L. Vanbever, "DITTO: WAN traffic obfuscation at line rate," in *Proc. NDSS*, Apr. 2022, p. 2.
- [58] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. NDSS*, San Diego, CA, USA, Feb. 2009, p. 2.
- [59] G. Cherubin, J. Hayes, and M. Juárez, "Website fingerprinting defenses at the application layer," *Proc. Privacy Enhanc. Technol.*, vol. 2017, no. 2, pp. 186–203, 2017. [Online]. Available: <https://petsymposium.org/popets/2017/popets-2017-0023.php>
- [60] E. Chan-Tin, T. Kim, and J. Kim, "Website fingerprinting attack mitigation using traffic morphing," in *Proc. ICDCS*, 2018, pp. 1575–1578.
- [61] S. Yu, G. Zhao, W. Dou, and S. James, "Predicted packet padding for anonymous Web browsing against traffic analysis attacks," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 4, pp. 1381–1393, Aug. 2012.
- [62] H. M. Moghaddam, B. Li, M. Derakhshani, and I. Goldberg, "SkypeMorph: Protocol obfuscation for ToR bridges," in *Proc. ACM CCS*, Raleigh, NC, USA, Oct. 2012, pp. 97–108, li2012.
- [63] W. Cui, J. Yu, Y. Gong, and E. Chan-Tin, "Realistic cover traffic to mitigate website fingerprinting attacks," in *Proc. IEEE ICDCS*, 2018, pp. 1579–1584.
- [64] X. Luo, P. Zhou, E. W. W. Chan, W. Lee, R. K. C. Chang, and R. Perdisci, "HTTPOS: Sealing information leaks with browser-side obfuscation of encrypted flows," in *Proc. NDSS*, 2011, p. 2.
- [65] S. Shan, A. N. Bhagoji, H. Zheng, and B. Y. Zhao, "A real-time defense against Website fingerprinting attacks," 2021. [Online]. Available: <https://arxiv.org/abs/2102.04291>
- [66] J. P. Degabriele, "Hiding the lengths of encrypted messages via Gaussian padding," in *Proc. ACM CCS*, ACM, 2021, pp. 1549–1565.
- [67] X. Liu, Z. Zhuo, X. Du, X. Zhang, Q. Zhu, and M. Guizani, "Adversarial attacks against profile HMM website fingerprinting detection model," *Cogn. Syst. Res.*, vol. 54, pp. 83–89, May 2019.
- [68] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *Proc. Security*, 2020, pp. 717–734.
- [69] R. Nithyanand, X. Cai, and R. Johnson, "GLOVE: A bespoke website fingerprinting defense," in *Proc. ACM WPES*, Scottsdale, AZ, USA, 2014, pp. 131–134.
- [70] T. T. T. Nguyen and G. Armitage, "A survey of techniques for Internet traffic classification using machine learning," *IEEE Commun. Surveys Tuts.*, vol. 10, no. 4, pp. 56–76, 4th Quart., 2008.
- [71] H. Kim, K. Claffy, M. Fomenkov, D. Barman, M. Faloutsos, and K. Lee, "Internet traffic classification demystified: Myths, caveats, and the best practices," in *Proc. ACM CoNEXT*, 2008, p. 11.
- [72] T. Karagiannis, K. Papagiannaki, and M. Faloutsos, "BLINC: Multilevel traffic classification in the dark," in *Proc. ACM SIGCOMM Comput. Commun.*, vol. 35, 2005, pp. 229–240.
- [73] R. Alshammari and A. N. Zincir-Heywood, "Can encrypted traffic be identified without port numbers, IP addresses and payload inspection?" *Comput. Netw.*, vol. 55, no. 6, pp. 1326–1350, 2011.
- [74] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. EuroS P*, Mar. 2016, pp. 439–454.
- [75] I. Akbari et al., "A look behind the curtain: Traffic classification in an increasingly encrypted Web," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 5, no. 1, pp. 1–26, Feb. 2021.
- [76] M. Korczyński and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *Proc. IEEE INFOCOM*, Apr. 2014, pp. 781–789.
- [77] Z. Xiong and N. Zilberman, "Do switches dream of machine learning? Toward in-network classification," in *Proc. ACM HotNets*, 2019, pp. 25–33.
- [78] B. M. Xavier, R. S. Guimarães, G. Comarela, and M. Martinello, "Programmable switches for in-network classification," in *Proc. IEEE INFOCOM*, 2021, pp. 1–10.
- [79] G. Siracusano and R. Bifulco, "In-network neural networks," 2018. [Online]. Available: <http://arxiv.org/abs/1801.05731>
- [80] D. Sanvito, G. Siracusano, and R. Bifulco, "Can the network be the AI accelerator?" in *Proc. ACM NetCompute*, 2018, pp. 20–25.
- [81] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, 2nd Quart., 2019.
- [82] T. Häilä-Järgensen, B. Ahlgren, P. Hurtig, and A. Brunström, "Measuring latency variation in the Internet," in *Proc. CoNEXT*, 2016, pp. 473–480.
- [83] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," IETF, RFC 8446, Aug. 2018. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8446>
- [84] A. Krogh, M. Brown, I. S. Mian, K. Sjölander, and D. Haussler, "Hidden Markov models in computational biology: Applications to protein modeling," *J. Mol. Biol.*, vol. 235, no. 5, pp. 1501–1531, 1994.
- [85] C. V. Wright, F. Monrose, and G. M. Masson, "On inferring application protocol behaviors in encrypted network traffic," *J. Mach. Learn. Res.*, vol. 7, pp. 2745–2769, Dec. 2006.
- [86] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*, 3rd ed. Amsterdam, The Netherlands: Morgan Kaufmann, 2011.
- [87] F. P. Miller, A. F. Vandome, and J. McBrewhster, *Levenshtein Distance: Information Theory, Computer Science, String (Computer Science), String Metric, Damerau Levenshtein Distance, Spell Checker, Hamming Distance*. London, U.K.: Alpha Press, 2009.
- [88] "Alexa—The Web information company," Accessed: May 14, 2020. [Online]. Available: <https://www.alexa.com/>
- [89] H. Chen, H. He, and A. Starr, "An overview of Web robots detection techniques," in *Proc. IEEE Cyber Security*, Dublin, Ireland, 2020, pp. 1–6.
- [90] W. John, M. Dusi, and K. Claffy, "Estimating routing symmetry on single links by passive flow measurements," ACM TRAC, Randburg, South Africa, Rep. TR 2010, Mar. 2010.
- [91] R. Weber, H.-J. Schek, and S. Blott, "A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces," in *Proc. VLDB*, 1998, pp. 194–205.
- [92] W. Zhang et al., "OpenNetVM: A platform for high performance network service chains," in *Proc. ACM HotMiddlebox*, Florianopolis, Brazil, 2016, pp. 26–31.
- [93] P. Emmerich, S. Gallenmüller, D. Raumer, F. Wohlfart, and G. Carle, "MoonGen: A Scriptable high-speed packet generator," in *Proc. IMC*, Tokyo, Japan, 2015, pp. 275–287.



Patrick Krämer (Member, IEEE) received the M.Sc. degree from the Technical University of Munich (TUM), Germany, in 2017. He joined the Chair of Communication Networks, TUM, in April 2017, where he is currently working as a Researcher and a Teaching Assistant. His research is focused on self-driving networks that learn to utilize the flexibility offered by modern networking technologies.



Benedikt Baier received the B.Sc. degree in electrical engineering and information technology from the Technical University of Munich (TUM), Germany, in 2020. In April 2020, he joined the Chair of Communication Networks, TUM. His research is focused on quantum networks and enhancing classical networks with quantum technologies.



Niklas Landerer received the B.Sc. degree in electrical engineering and information technology from the Technical University of Munich (TUM) in 2020, where he is currently pursuing the master's degree. In 2020, he joined the Chair of Communication Networks, TUM, as a Research Assistant.



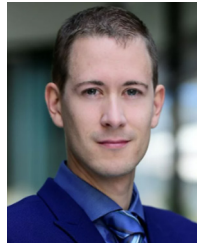
Andreas Blenk received the Doktor-Ingenieur (Dr.-Ing.) degree (summa cum laude) from Technische Universität München (TUM) in May 2018. He is a Research Scientist with Siemens AG, where he has been a part of the Industrial Networks and Wireless Group (T CED INW-DE) since April 2022. In his current role, he is focused on the automation, measurement, and validation of industrial networks and is involved in defining and writing blueprints for the Siemens AG networking portfolio. Prior to joining Siemens AG, he was a member of the Chair of Communication Networks, led by Prof. W. Kellerer, at TUM beginning in June 2012. From 2018 to 2022, he continued his work at TUM as a Postdoctoral Fellow, further honing his expertise in the field of communication networks. During this time, he also served as a Senior Research Fellow with the Communication Technologies Group, Faculty of Computer Science, University of Vienna from March 2019 to January 2022.



Philip Diederich (Graduate Student Member, IEEE) received the B.Sc. degree in electrical engineering and information technology from the Technical University of Munich (TUM), Germany, in 2019. He joined the Chair of Communication Networks, TUM, in November 2021. His research is focused on software-defined networks and resource management in network function virtualization.



Alexander Griessel (Graduate Student Member, IEEE) received the Bachelor of Science and Master of Science degrees in electrical and computer engineering from the Technical University of Munich in 2020 and 2022, respectively, where he is currently pursuing the Ph.D. degree with the Chair of Communication Networks, specializing in resource allocation mechanisms in semantic communication.



Martin Mieth received the M.B.A. degree and the Ph.D. degree in electrical engineering and information technology. As the Vice President of Engineering, he is responsible for all aspects of ipoque's OEM network analytics solutions with a strong focus on innovation and research partnerships with universities. Before his current position, he successfully led the Deep Packet Inspection Development Department and established an in-house research team dedicated to ML and AI. He can rely on more than nine years of managing experience for several specialized teams, all while applying agile values and methods. Before joining ipoque, he worked for the German Federal Armed Forces as an IT Officer in charge of numerous software development and IT projects.



Oliver Hohlfeld received the B.Sc. and M.Sc. degrees in computer science from the Darmstadt University of Applied Sciences, Institute Eurecom, and the Darmstadt University of Technology, and the Ph.D. (Dr. rer. nat.) degree from TU Berlin. He is a Full Professor with the University of Kassel, where he heads the Distributed Systems Group. Before, he was a Full Professor with the Brandenburg University of Technology, heading the Chair of Computer Networks. Before, he was with RWTH Aachen University and with TU Berlin/Deutsche

Telekom Innovation Laboratories. He was a Visiting Scholar with the Group of Paul Barford, University of Wisconsin–Madison, USA.



Wolfgang Kellerer (Senior Member, IEEE) is a Full Professor with the Technical University of Munich, heading the Chair of Communication Networks, Department of Electrical and Computer Engineering. Before, he was for over ten years with NTT DOCOMO's European Research Laboratories. He currently serves as an Associate Editor for IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT and an Area Editor for Network Virtualization for IEEE COMMUNICATIONS SURVEYS AND TUTORIALS.